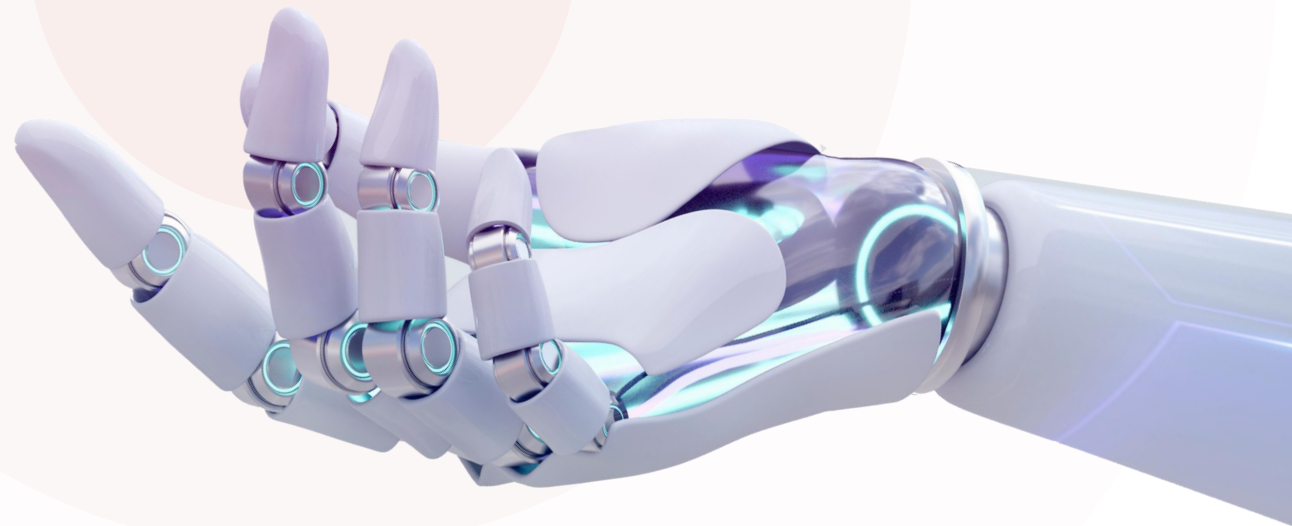


Python核心语法

AI Copilot

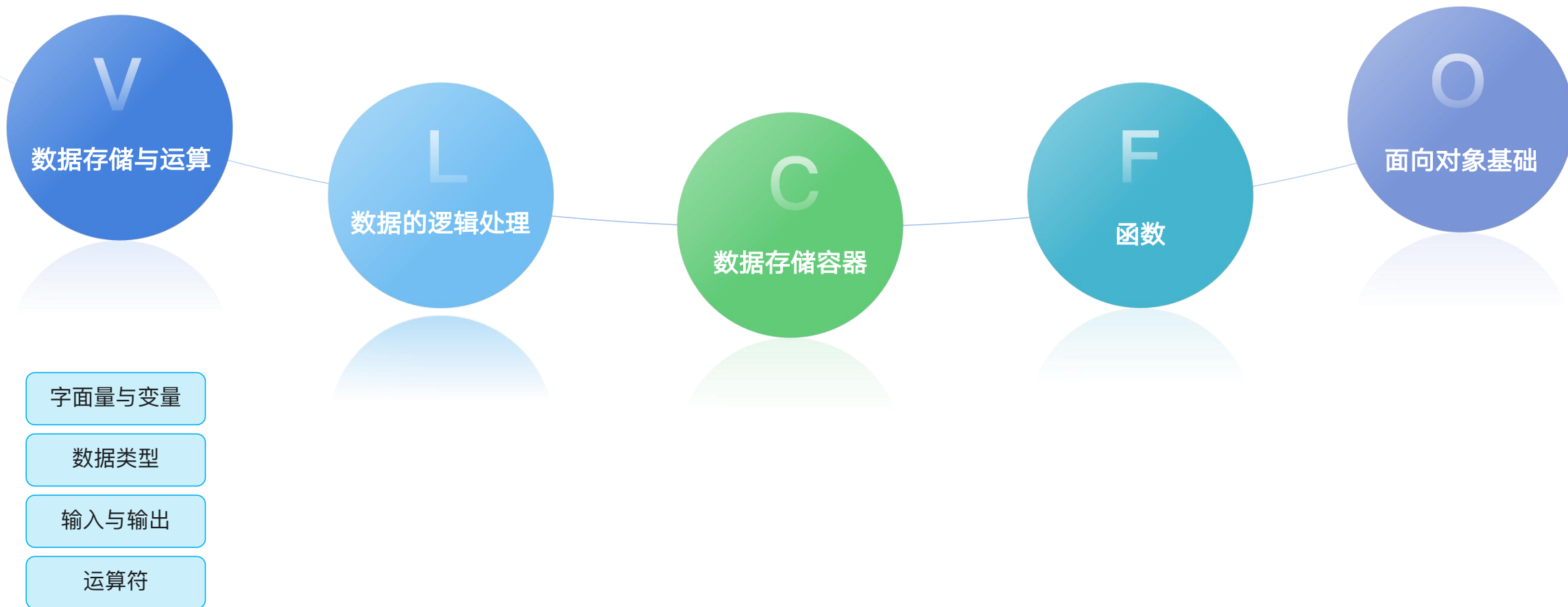


多一句没有，少一句不行，用更短的时间，教会更实用的技术



Python核心语法

多一句没有，少一句不行，用更短的时间，教会更实用的技术



```
# 案例：银行卡ATM取款

# 总金额
total = 10000

# 1. 输入密码
password = input("请输入您的银行卡密码： ")
print(f"密码正确， {password}")

# 2. 输入取款金额
num = input("请输入您的取款金额： ")

# 3. 计算余额并输出 --> num 转为 int类型 ---> int(..)
print(f"取款后银行卡余额为： {total - int(num)}")
```

从上往下，逐行执行

扫二维码登录



请使用 [哔哩哔哩客户端](#)
扫码登录或扫码下载APP

密码登录 | 短信登录

账号 18888888888

密码 [忘记密码?](#)

注册

登录

其他方式登录

 微信登录

 微博登录

 QQ登录

未注册过哔哩哔哩的手机号，我们将自动帮你注册账号
登录或完成注册即代表你同意 [用户协议](#) 和 [隐私政策](#)

- 如果 账号及密码 正确，则登录成功，进入B站首页
- 否则，登录失败，提示错误信息

条件判断

首页 番剧 直播 游戏中心 会员购 漫画 赛事 S15 下载客户端

黑马程序员python

大会员 消息 动态 收藏 历史 创作中心 投稿





动态

热门

番剧

国创

综艺

动画

鬼畜

舞蹈

娱乐

科技数码

美食

汽车

体育运动

vlog

专栏

活动

社区中心

电影

电视剧

纪录片

游戏

音乐

影视



仙帝归来风云起，新旧旧敌皆尘埃!

不会作
到理解

922

非科班从不会
才是入门AI最
跟着迪哥学

亲表弟找我借
冷血点可以避

4.6万

有趣的小敏

播放器模式

罗生门 (Follow)

专辑: 罗生门 (Follow) 歌手: 梨冻紧 / Wiz_H... 来源: 热歌榜

歌词 百科 相似推荐

作曲: 梨冻紧 / Wiz_H张子豪

编曲: Wiz_H张子豪

钢琴: 梨冻紧

封面: 蓝明初

混音: 官硕

母带: Mai " No Label Crew "

监制: Mai " No Label Crew "

发行: No Label Crew

企划: 苏文媛 张琰

00:03 / 04:03

10w+

循环

词

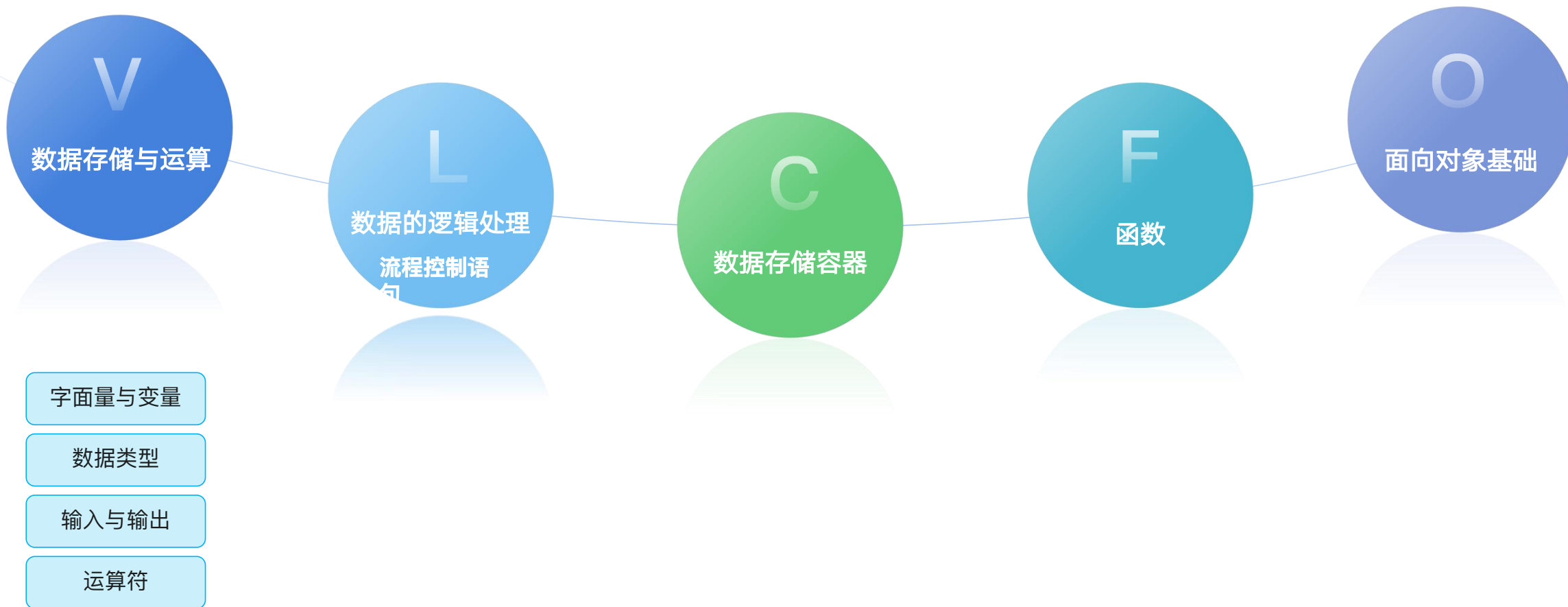
极高

循环



Python核心语法

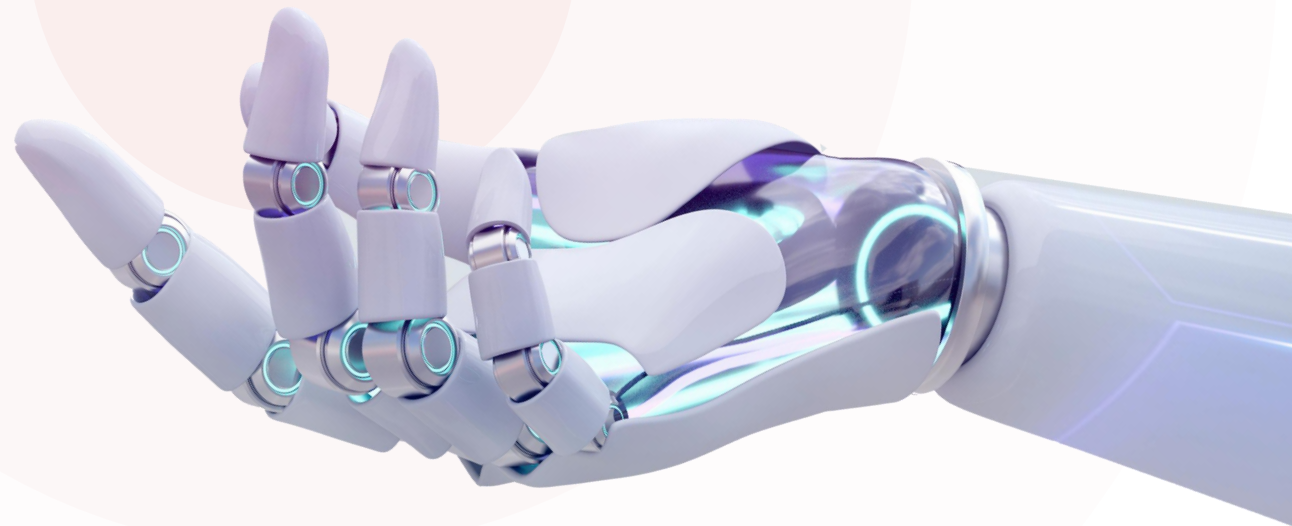
多一句没有，少一句不行，用更短的时间，教会更实用的技术



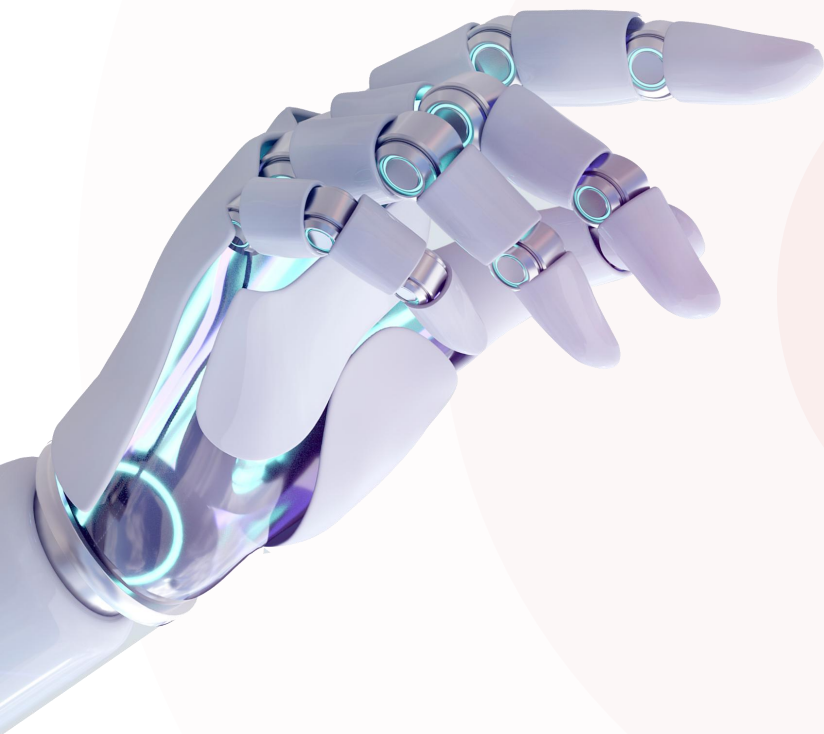
Python核心语法

——流程控制语句

AI Copilot

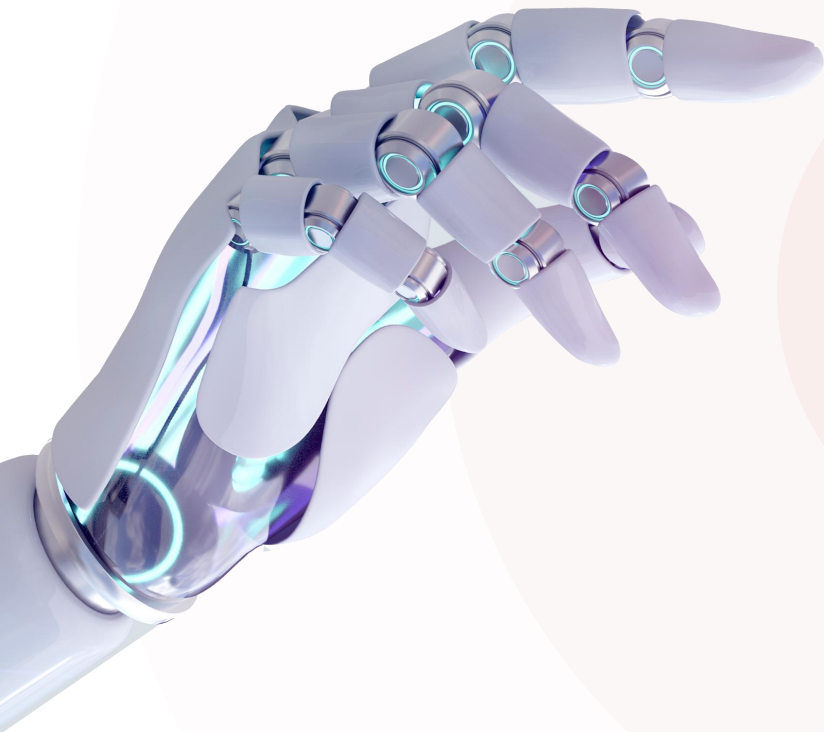


多一句没有，少一句不行，用更短的时间，教会更实用的技术



CONTENT 目录

- ◆ 条件判断
- ◆ 模式匹配
- ◆ 循环

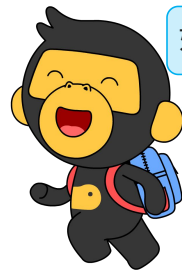


1

条件判断

- ◆ if语句基本格式
- ◆ if语句进阶
- ◆ 案例

if条件判断



如果我的高考分数超过680，我就去清华读书。

- 场景：只有满足指定条件，才会执行对应的代码逻辑。

```
# if条件判断基本格式
```

```
if 要判断的条件:
```

```
    条件成立时，要执行对应的操作
```

判断语句的结果，必须是布尔类型True或False

True 会执行if内的代码

False则不会执行

if条件判断基本格式

```
# 定义变量
```

```
score = 695
```

```
# 条件判断
```

```
if score > 680:
```

```
    print("欢迎你, 来清华读书")
```

```
# 定义变量
```

```
score = 695
```

```
# 条件判断
```

```
if score > 680:
```

```
    print("欢迎你, 来清华读书")
```

```
    print("也恭喜你即将踏入精彩的大学生活")
```

注意：Python中是通过缩进来描述代码的归属的，归属于if代码块的语句，需要在前方缩进4个空格(按Tab键Pycharm会自动转为空格)。

BRIEF 小结

1. if条件判断的基本格式

```
if 要判断的条件:
```

```
    条件成立时，要执行对应的操作1
```

```
    条件成立时，要执行对应的操作2
```

2. if语句的注意事项

- 判断条件的结果一定要是布尔类型
- 不要忘记判断条件后的冒号(:)
- if语句里面的代码块，需要在前方缩进空格（建议4个空格），通过缩进来描述代码的层级关系（归属）

案例

输入账号密码后登录B站



- 需求：结合前面学习的输入输出及if条件判断的知识，完成B站登录功能的实现（正确账号和密码为18888888888/666888）。
1. 提示用户输入账号和密码。
 2. 判断账号和密码是否正确，如果账号和密码都正确，则登录成功，进入B站首页。
 3. 如果账号和密码有一个错误，则登录失败，提示错误信息。

扫描二维码登录

请使用 [哔哩哔哩客户端](#)
扫码登录或扫码下载APP

密码登录 | 短信登录

账号 18888888888

密码 [忘记密码?](#)

注册 登录

其他方式登录

微信登录 微博登录 QQ登录

未注册过哔哩哔哩的手机号，我们将自动帮你注册账号
登录或完成注册即代表你同意 [用户协议](#) 和 [隐私政策](#)

思考

```
# B站登录
ok_account = "188888888888"
ok_password = "666888"

account = input("请输入B站账号: ")
password = input("请输入B站密码: ")

if account == ok_account and password == ok_password:
    print("登录成功 ~")
    print("进入B站首页 ~")

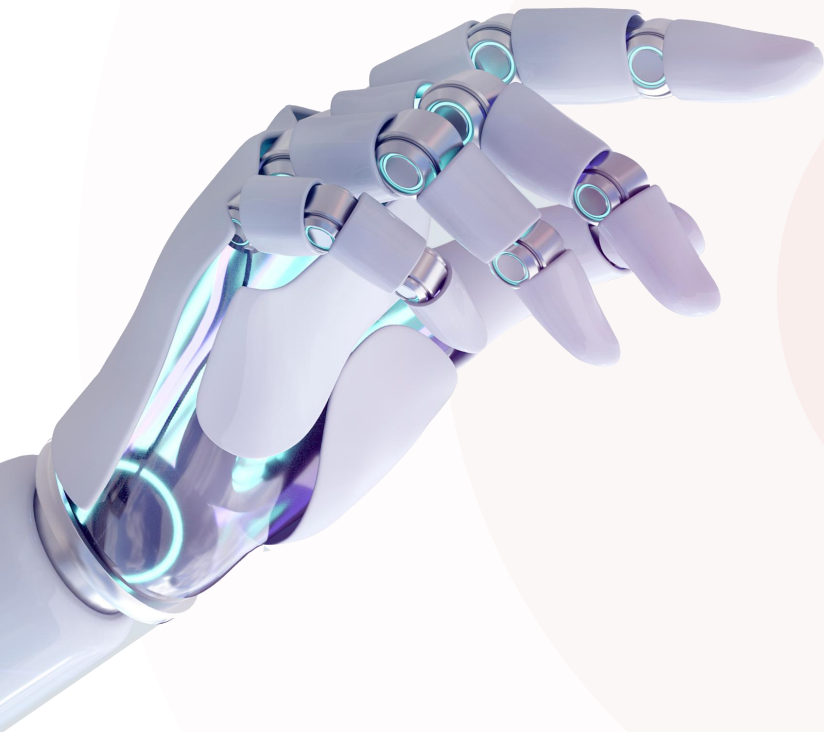
if account != ok_account or password != ok_password:
    print("登录失败 !")
    print("用户名或密码错误 !")
```

登录成功

登录失败

繁琐、效率不高

用户名和密码都正确，登录成功；否则，登录失败



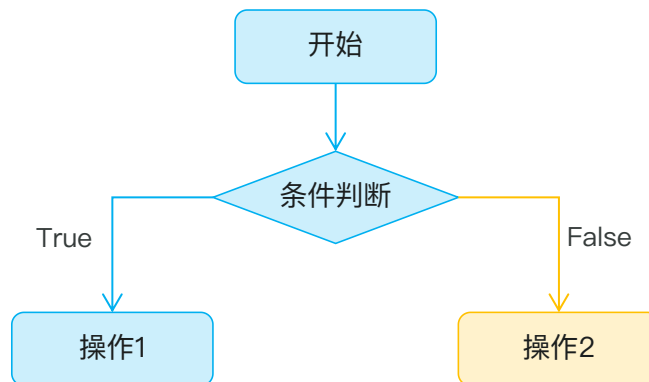
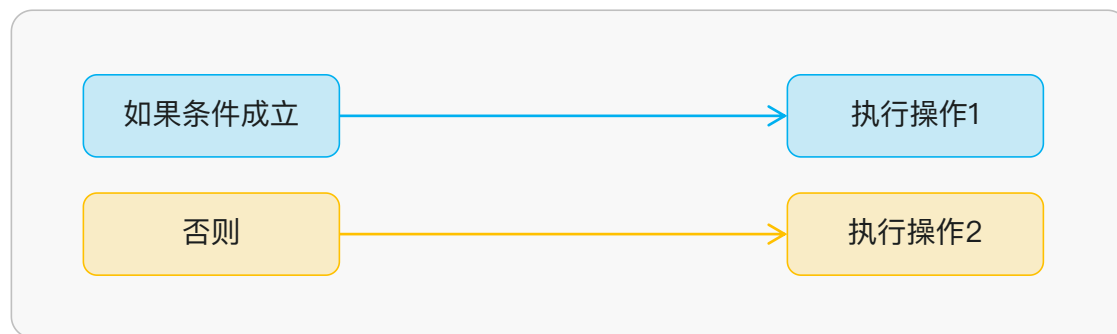
1

条件判断

- ◆ if语句的基本格式
- ◆ if语句进阶
- ◆ 案例

if语句进阶-

多一句没有，少一句不行，用更短的时间，教会更实用的技术



```
# if...else 结构
```

```
if 要判断的条件:
```

```
    条件成立时，执行对应的操作1
```

```
else:
```

```
    条件不成立时，执行的操作2
```




案例

完成如下需求



1. 需求：根据用户输入的年份，判断这一年是闰年还是平年。

- 非整百年份，且能被4整除的年份是闰年
- 整百年份（如1900年、2000年）必须能被400整除才是闰年

BRIEF 小结

1. if...else 语句的语法

if 要判断的条件:

条件成立时，执行对应的操作1

else:

条件不成立时，执行的操作2

2. if...else 语句注意事项

- else是不需要条件判断的，当if条件不成立时，else就执行了
- else代码块，也需要使用空格缩进（建议4个空格）



练习

完成如下需求



1. 需求1：根据用户输入的数字，判断该数字是奇数还是偶数。
2. 需求2：根据用户输入的年龄，判断该用户是否已经成年 (≥ 18 ，成年；否则，未成年)。
3. 需求3：根据用户输入的数字，判断该数字是正数还是负数（不考虑0）。
4. 需求4：根据用户输入的考试分数，判断该分数是否及格了（大于等于60就是及格了）。

思考

- 根据用户输入的数字，判断该数字是正数，还是负数，还是0？
 - 如果 数字 > 0 : 正数
 - 如果 数字 < 0 : 负数
 - 否则: 0

if 要判断的条件:

条件成立时，执行对应的操作1

else:

条件不成立时，执行的操作2

不满足

if语句进阶-if...elif...else

- 根据用户输入的数字，判断该数字是正数，还是负数，还是0？
 - 如果 数字 > 0：正数
 - 如果 数字 < 0：负数
 - 否则：0

if...elif...else结构

if 要判断的条件1:

条件成立时，执行对应的操作1

①

elif 要判断的条件2:

条件成立时，执行对应的操作2

②

else:

条件不成立时，执行的操作3

③

if...elif...else结构

num = int(input("请输入数字: "))

if num > 0:

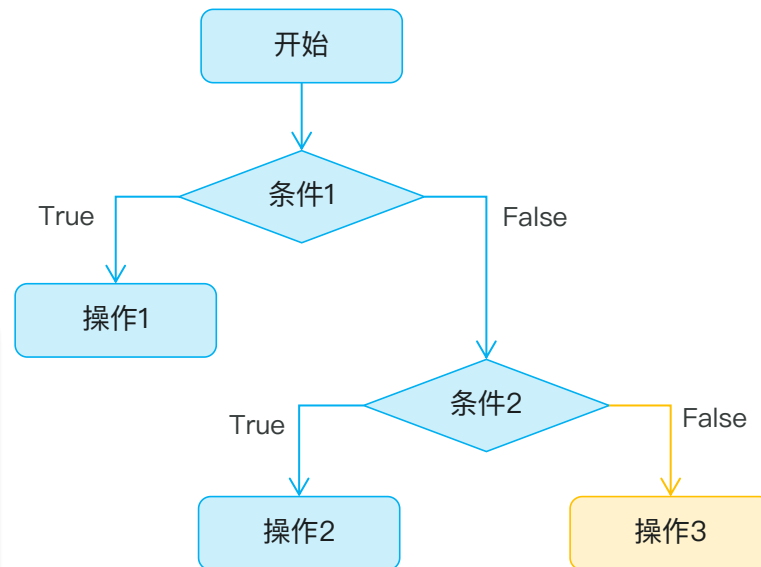
print(f"{num} 是 正数")

elif num < 0:

print(f"{num} 是 负数")

else:

print(f"{num} 是 0")



注意：满足第1个条件，就不会判断第2个部分和第3个部分；第1个条件不成立，才会判断第2个条件；1和2都不成立，才会进入else。



案例

完成如下需求



1. 根据输入用户名、密码进行登录系统。

- 用户名、密码为 admin/666888 或 root/547527 或 zhangsan/123456，则输出登录成功
- 否则就提示用户名或密码错误

BRIEF 小结

1. if...elif...else语句的作用

- 可以完成多个条件的判断(多分支判断)

2. if...elif...else语句注意事项

- elif 可以写多个
- else 可有可无，如果有必须放在最后
- 多个条件判断是有顺序的，从上到下判断，如果前面有条件成立了，后面的条件不会判断了



练习

完成如下需求

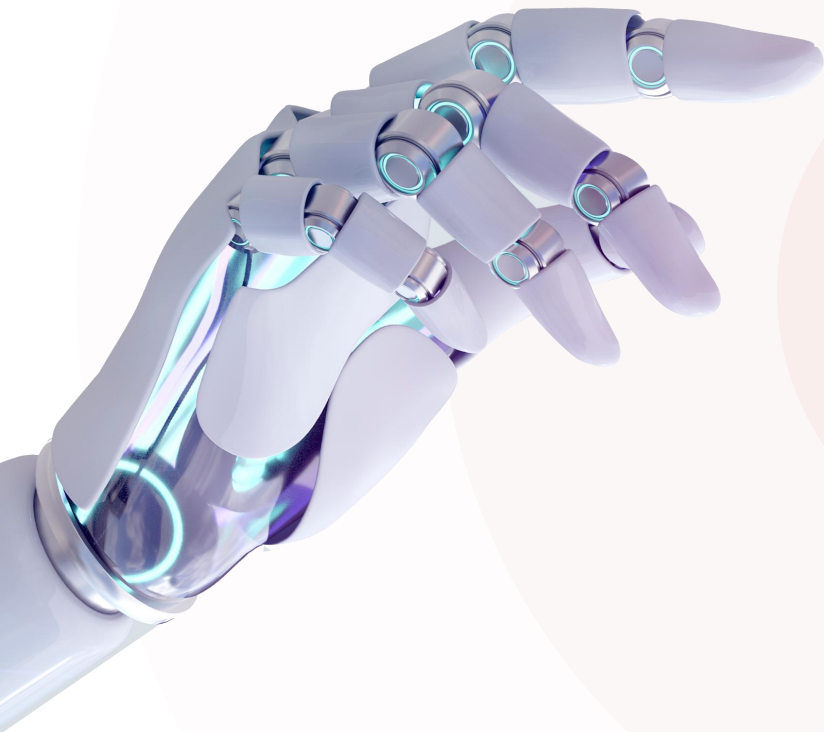


1. 根据输入的考试成绩，判断成绩等级。

- 大于等于85分为优秀
- 60-85分为及格
- 否则就是不及格

2. 购物折扣计算：根据输入的购物车的商品总额，以及如下的折扣规则，计算实际应付的金额。

- 金额 ≥ 500 : 8折
- $300 \leq \text{金额} < 500$: 9折
- $100 \leq \text{金额} < 300$: 95折
- 金额 < 100 : 无折扣



1

条件判断

- ◆ if语句的基本格式
- ◆ if语句进阶
- ◆ 案例

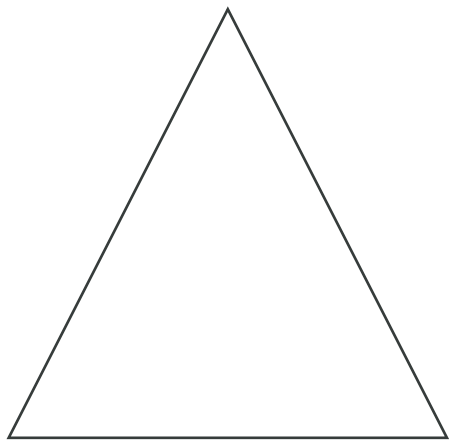


案例

完成如下需求



- 三角形类型判断：根据输入的三个边的边长(正整数)，判定是等边三角形、等腰三角形、普通三角形，还是不能构成三角形。
 - 构成三角形的条件：两边之和大于第三边
 - 三角形判定规则：
 - 三个边都相等: 等边三角形
 - 两个边相等: 等腰三角形
 - 三个边都不相等: 普通三角形





练习

完成如下需求

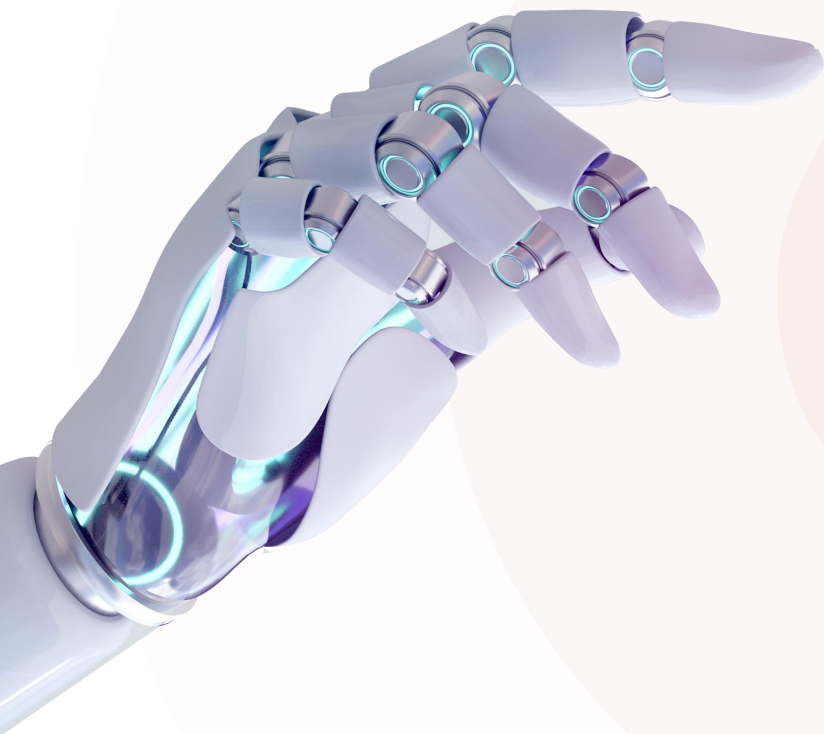


- 北京市居民年度用电电费计算：根据输入的用电度数，计算电费
 - 北京市居民电费采用阶梯电价计价方式，所谓阶梯电价是指按照用户消费的电量分段定价，用电价格随用电量增加呈阶梯状逐级递增的一种电价定价机制。
 - 阶梯电价规则：
 - 第一档：2880度以下，电费单价0.4883元/度
 - 第二档：2880–4800度，电费单价0.5383元/度
 - 第三档：4800度以上，电费单价0.7883元/度

QUESTION
思考

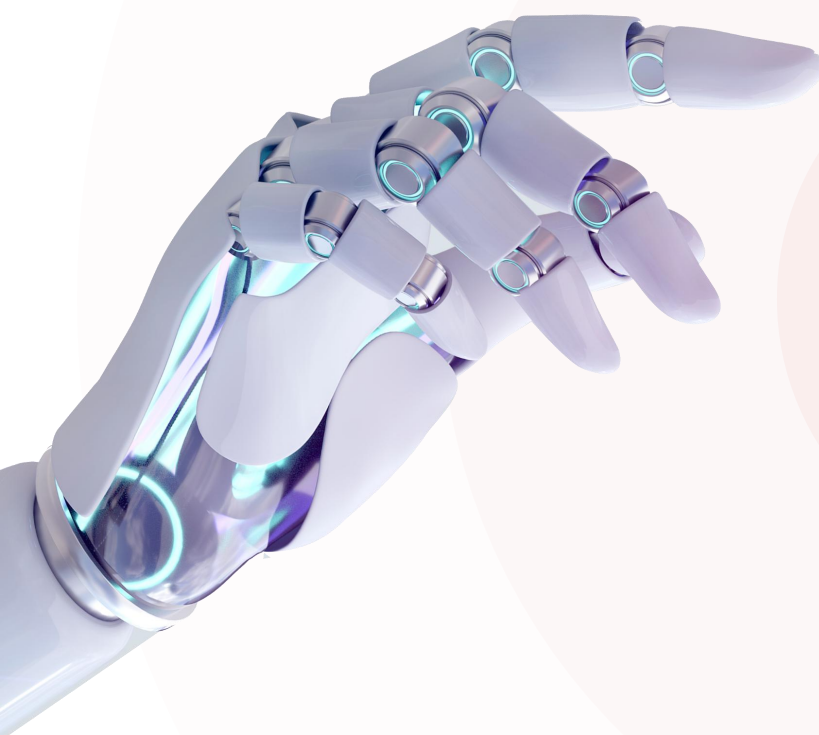
```
# 工作日程安排
day = input("请输入星期几(1-7): ")
if day == "1":
    print("周一：工作会议日")
elif day == "2":
    print("周二：学习培训日")
elif day == "3":
    print("周三：项目开发日")
elif day == "4":
    print("周四：代码审查日")
elif day == "5":
    print("周五：总结规划日")
elif day == "6" or day == "7":
    print("周末：休息放松")
else:
    print("输入错误")
```

重复
繁琐



CONTENT 目录

- ◆ 条件判断
- ◆ 模式匹配
- ◆ 循环



2

模式匹配

模式匹配 match...case

- 结构模式匹配就是用一个清晰的 模板 去精准的匹配数据的结构和内容，匹配成功则执行响应的操作。

```
# 工作日程安排
day = input("请输入星期几(1-7): ")
if day == "1":
    print("周一：工作会议日")
elif day == "2":
    print("周二：学习培训日")
elif day == "3":
    print("周三：项目开发日")
elif day == "4":
    print("周四：代码审查日")
elif day == "5":
    print("周五：总结规划日")
elif day == "6" or day == "7":
    print("周末：休息放松")
else:
    print("输入错误")
```

```
# 工作日程安排
day = input("请输入星期几(1-7): ")
match day:
    case "1":
        print("周一：工作会议日")
    case "2":
        print("周二：学习培训日")
    case "3":
        print("周三：项目开发日")
    case "4":
        print("周四：代码审查日")
    case "5":
        print("周五：总结规划日")
    case "6" | "7":
        print("周末：休息放松")
    case _:
        print("输入错误")
```

模式匹配 match...case

```
# 工作日程安排
day = input("请输入星期几(1-7): ")
match day:
    case "1":
        print("周一：工作会议日")
    case "2":
        print("周二：学习培训日")
    case "3":
        print("周三：项目开发日")
    case "4":
        print("周四：代码审查日")
    case "5":
        print("周五：总结规划日")
    case "6" | "7":
        print("周末：休息放松")
    case _:
        print("输入错误")
```

执行流程：

1. 首先计算match指定的表达式的值。
2. 从上到下依次和case后面的值进行匹配，匹配正确，就执行对应语句。
3. 如果前面所有的case都没有匹配上，就会执行默认的案例 _。

→ 其中的 | 表示或的关系，匹配多个模式中的任意一个

→ 匹配其他所有情况



案例

简易计算器



- 实现一个计算器，可以实现+ - * / 运算，用户输入需要运算的两个数以及运算符之后，就可以进行计算。

小结

1. match...case语法

match 表达式:

case 值1:

操作1

case 值2 if 条件表达式:

满足条件且值为值2

操作2

case 值3 | 值4 :

匹配值3或值4

操作4

case _:

其他情况

默认操作

3.10版本中的新语法

2. match...case应用场景

- match: 基于某个变量的多个固定值进行分支判断时，可以使用match模式匹配
- if: 条件判断涉及复杂的逻辑判定、范围比较及组合条件时

练习

简单游戏指令系统



请你编写一个游戏角色移动控制系统，根据玩家输入的不同指令，控制游戏角色执行相应的动作(输出控制台)。

具体规则：

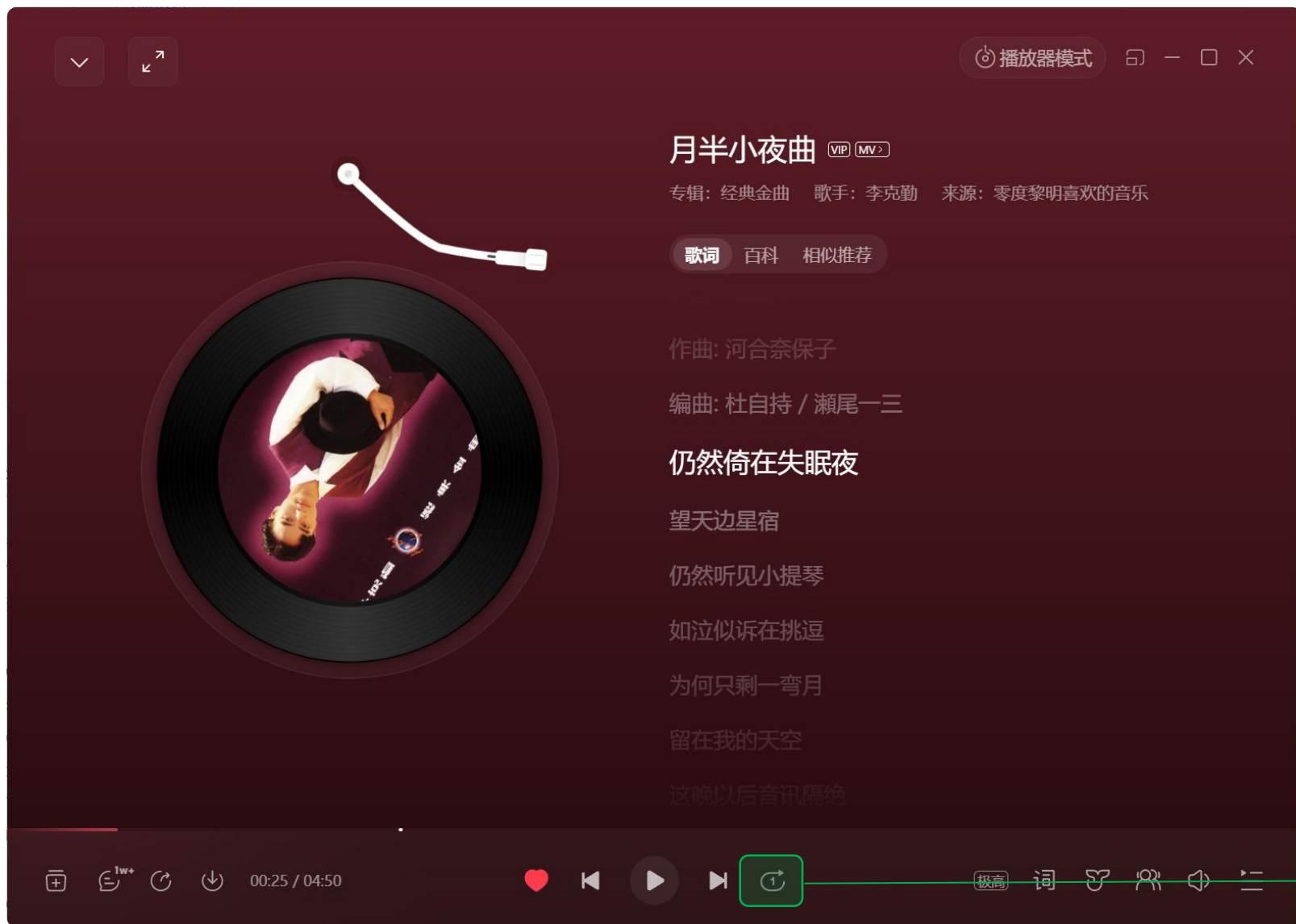
玩家输入	对应动作
上 / w / W	角色向上移动
下 / s / S	角色向下移动
左 / a / A	角色向左移动
右 / d / D	角色向右移动
跳 / " "(空格)	角色跳跃
攻击 / j / J	角色发动攻击
退出 / esc / ESC	角色退出游戏

CONTENT
目录

- ◆ 条件判断
- ◆ 模式匹配
- ◆ 循环

循环

多一句没有，少一句不行，用更短的时间，教会更实用的技术



循环：重复的执行某件事情

单曲循环 / 列表循环

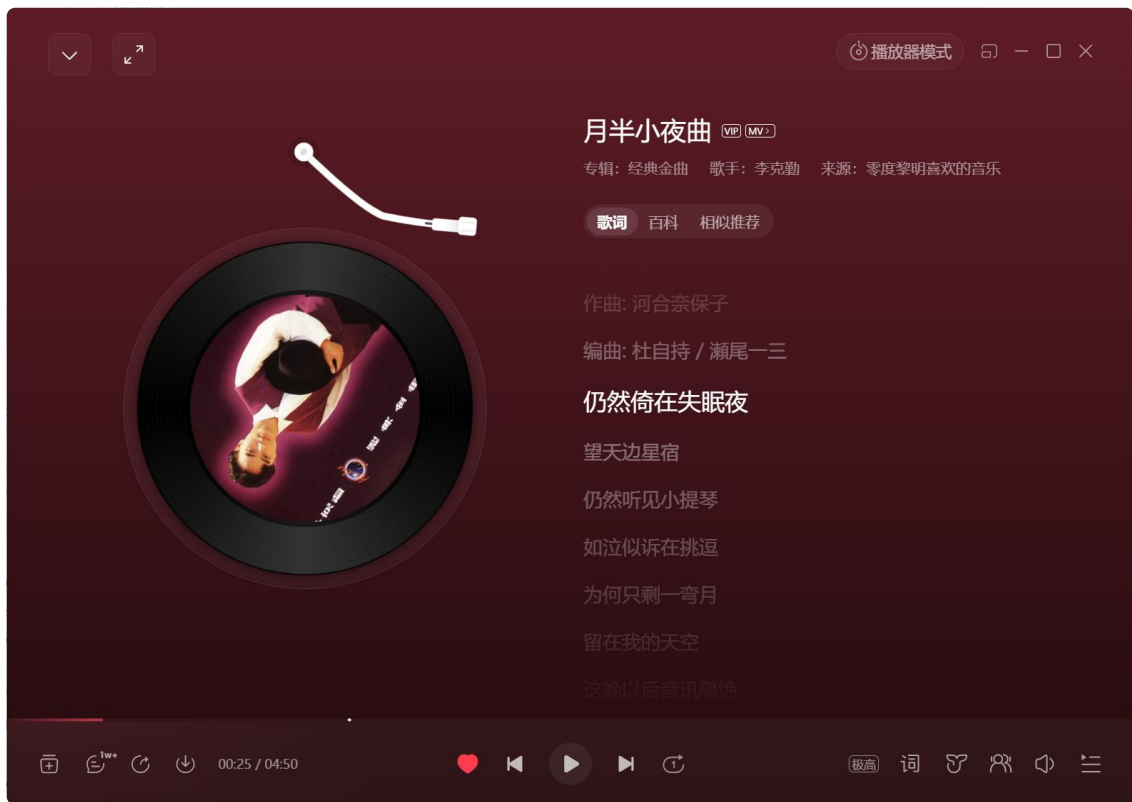


循环

多一句没有，少一句不行，用更短的时间，教会更实用的技术



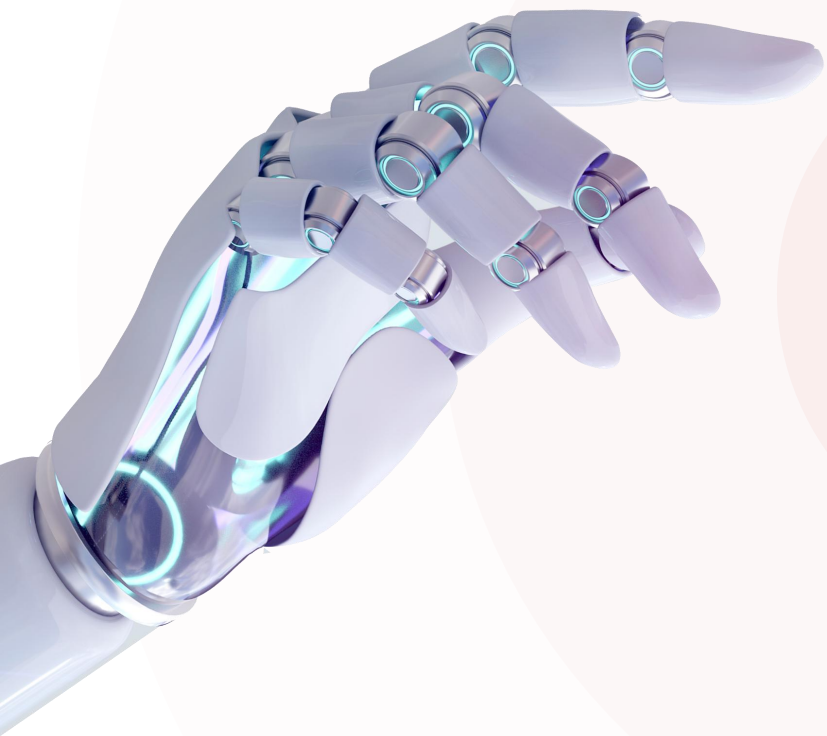
循环：重复的执行某件事情



while循环



for循环



3

循环

- ◆ while循环
- ◆ for循环
- ◆ 嵌套循环
- ◆ 案例



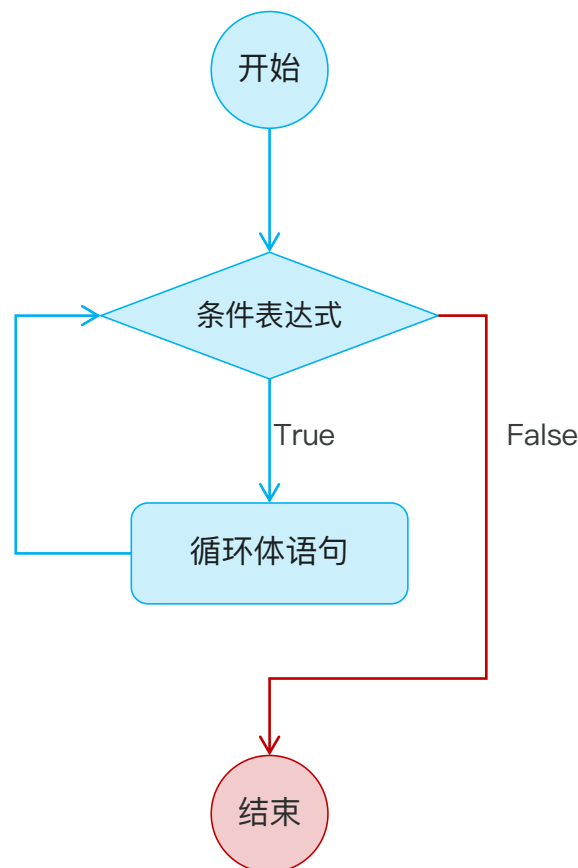
while

- while循环的语法结构：

```
# while语法结构
while 条件表达式:
    循环体语句1
    循环体语句2
    ...
```

```
# while语法结构
while 条件表达式:
    循环体语句1
    循环体语句2
    ...
else:
    条件为False,循环正常结束时执行
```

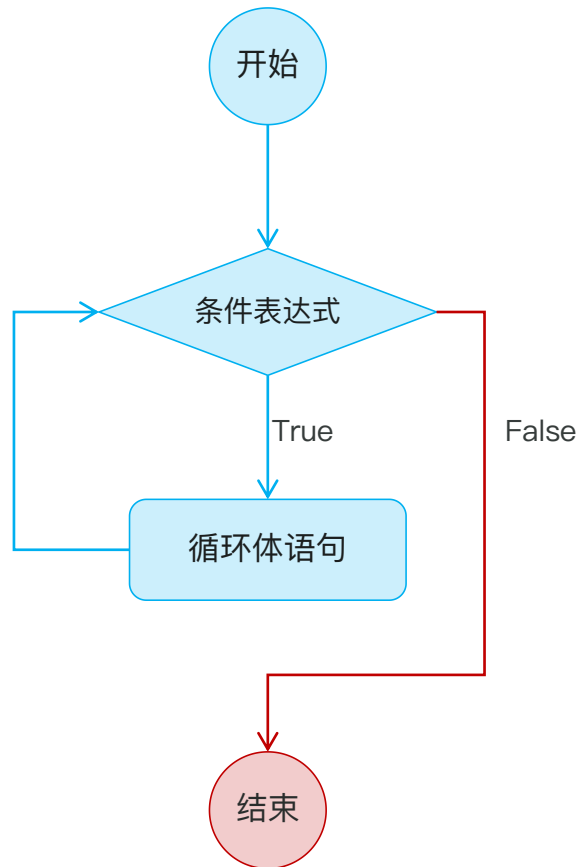
- while循环的执行流程：



注意：while循环是通过条件表达式，来控制是否要进行下一次循环。

while

- while循环的执行流程：



打印10遍 "人生苦短，我用Python~"

```
i = 0
while i < 10:
    print("人生苦短, 我用Python~")
    i += 1
else:
    print("循环正常结束, 执行完毕")
```

→ 循环的条件

→ 条件成立时，执行的循环体逻辑

→ 条件不成立循环结束时，执行的代码

只要条件满足，会无限循环执行

BRIEF 小结

1. while循环的语法格式

`while` 条件表达式:

条件成立时，执行的操作（循环体语句）

条件成立时，执行的操作（循环体语句）

.....

`else:`

循环正常结束时，执行的操作

可选

2. while循环的注意事项

- 条件表达式的结果为布尔类型
- 通过空格缩进表述层级关系
- 规划好循环终止的条件，否则将无限循环（死循环）

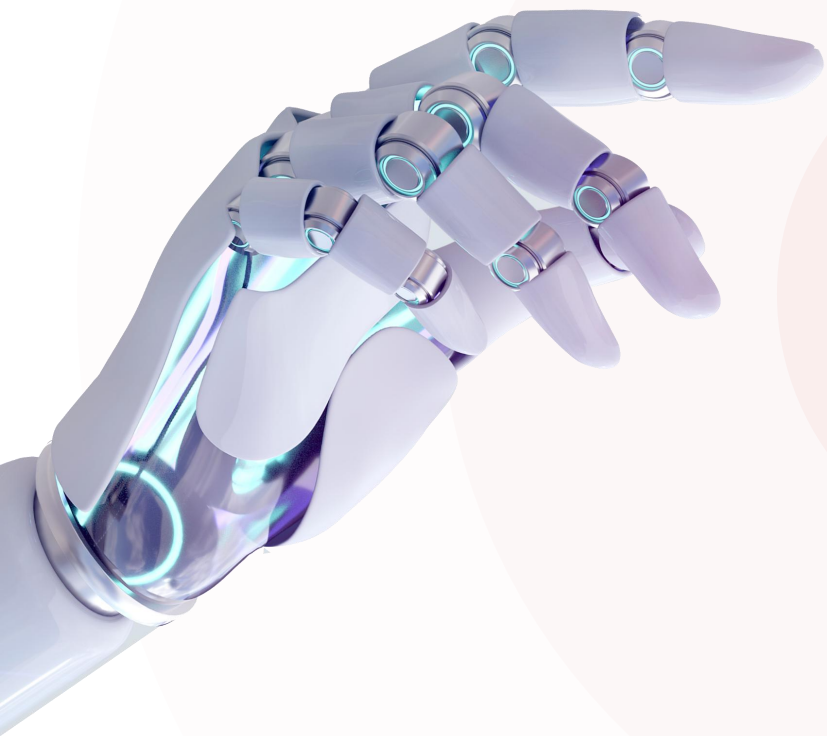


案例

计算1-100之间所有偶数的累加之和。



- 思考1：如何获取到1-100之间的偶数？
- 思考2：累加运算使用什么运算符？



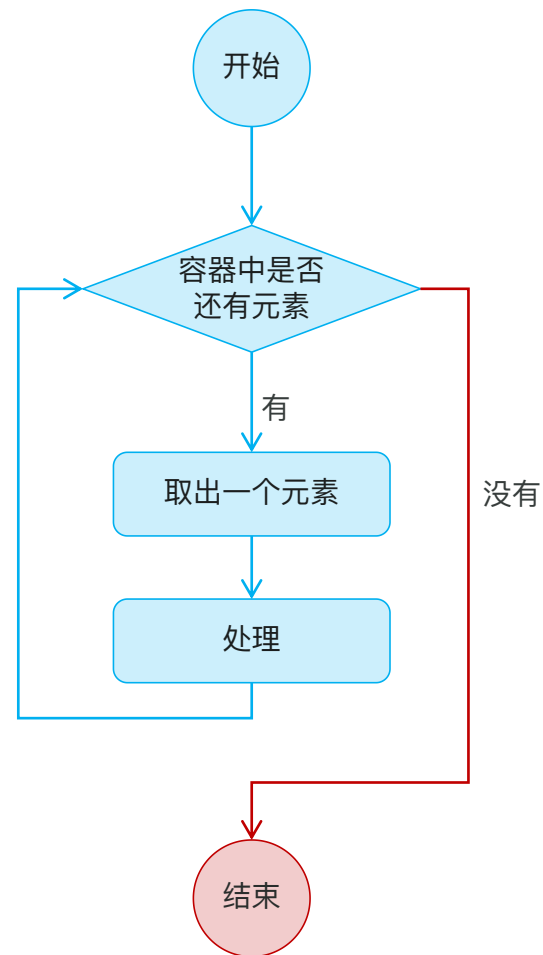
3

循环

- ◆ while循环
- ◆ for循环
- ◆ 嵌套循环
- ◆ 案例

for循环

- while循环是通过条件表达式来控制是否要进行下一次循环的。而for循环，本质是一种**轮询遍历**机制，对一批内容进行逐个处理。

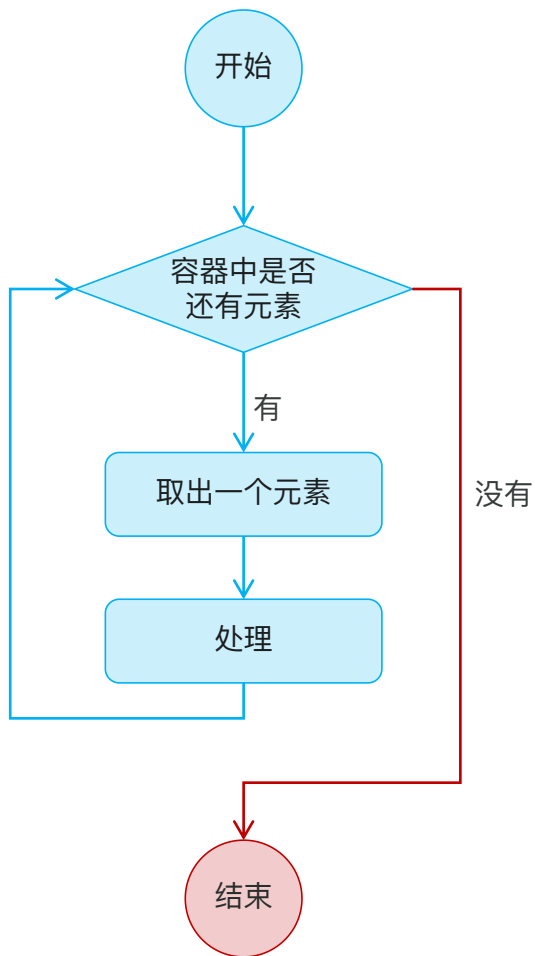




for循环

多一句没有，少一句不行，用更短的时间，教会更实用的技术

- while循环是通过条件表达式来控制是否要进行下一次循环的。而for循环，本质是一种**轮询遍历**机制，对一批内容进行逐个处理。



```
# for循环结构
for 元素 in 待处理数据集:
    循环体代码(对元素进行处理)
else:
    循环结束时, 执行的代码
```

可选

遍历字符串 "Hello-Python"

```
# 定义要遍历的字符串
msg = "Hello-Python"

# 遍历字符串,并处理
for i in msg :
    print(i)
else:
    print("循环结束")
```



for循环

多一句没有，少一句不行，用更短的时间，教会更实用的技术

```
# 定义要遍历的字符串
```

```
msg = "Hello-Python"
```

```
# 遍历字符串,并处理
```

```
for i in msg:
```

```
    print(i)
```

```
else:
```

```
    print("循环结束")
```

H	e	l	l	o	-	P	y	t	h	o	n
---	---	---	---	---	---	---	---	---	---	---	---

BRIEF 小结

1. for循环的语法格式

```
for 元素 in 待处理数据集:  
    循环体代码(对元素进行处理)
```

```
else:
```

不满足循环条件时，执行的操作

可选

2. for循环与while循环的场景

- while循环：用于在某个条件满足时一直循环，循环的次数通常是未知的，只知道循环开始/结束的条件。（关注的是循环的条件）
- for循环：用于对一个已知的数据集进行遍历或已知次数的循环。（关注的是遍历每一个元素）



案例

基于for循环完成如下需求



1. 计算1-100之间所有奇数之和。
2. 计算100-500之间所有3的倍数的数字之和。

for 元素 in 待处理数据集:

循环体语句(对元素进行处理)



如果获取到1-100之间的数据集??

range语句

range语句

- **作用：**生成指定规则的数字序列
- **用法1：** `range(end)` -> 获取一个从0开始，到end结束的数字序列（不含end本身）
 - `range(5)`获取的数据就是 0,1,2,3,4
- **用法2：** `range(start,end)` -> 获取一个从start开始，到end结束的数字序列（不含end本身）
 - `range(2,8)`获取的数据就是 2,3,4,5,6,7
- **用法3：** `range(start,end,step)` -> 获取一个从start开始，到end结束的数字序列，step步长（不含end本身）
 - `range(0,10,2)`获取的数据就是 0,2,4,6,8



案例

基于for循环完成如下需求

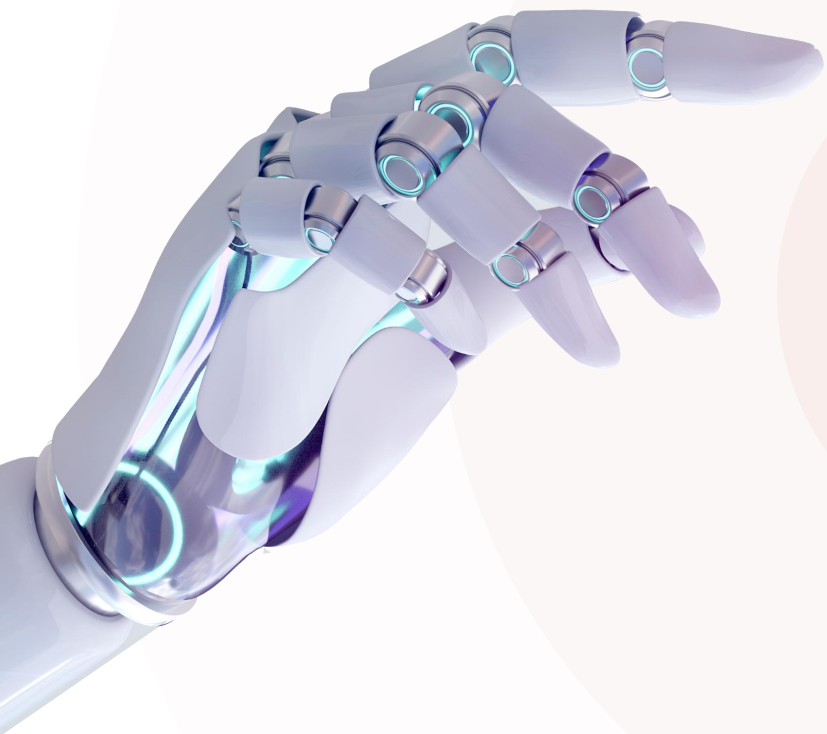


1. 计算1-100之间所有奇数之和。
2. 计算100-500之间所有3的倍数的数字之和。

BRIEF 小结

1. range(..)语句的作用是什么？

- 生成指定规则的数字序列
- 用法：
 - range(end)
 - range(start,end)
 - range(start,end,step)



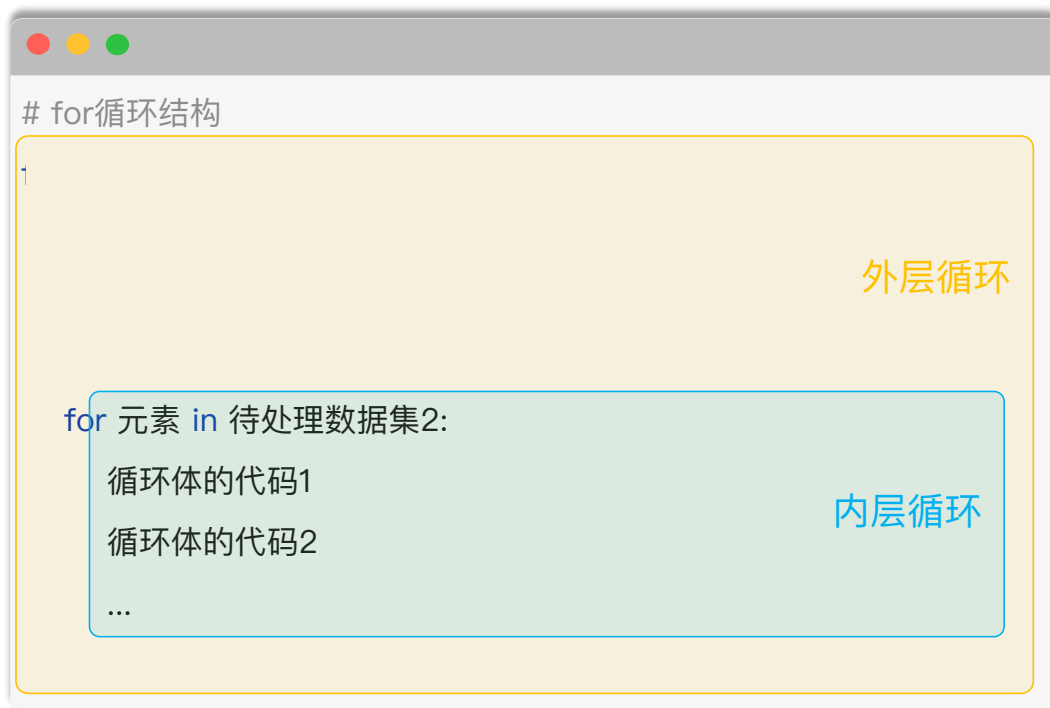
3

循环

- ◆ while循环
- ◆ for循环
- ◆ 嵌套循环
- ◆ 案例

嵌套循环

- 语法：



打印一个长度为m，宽度为n的长方形

```
* * * * *  
* * * * *  
* * * * *  
* * * * *
```



案例

打印99乘法表



1 x 1 = 1									
1 x 2 = 2	2 x 2 = 4								
1 x 3 = 3	2 x 3 = 6	3 x 3 = 9							
1 x 4 = 4	2 x 4 = 8	3 x 4 = 12	4 x 4 = 16						
1 x 5 = 5	2 x 5 = 10	3 x 5 = 15	4 x 5 = 20	5 x 5 = 25					
1 x 6 = 6	2 x 6 = 12	3 x 6 = 18	4 x 6 = 24	5 x 6 = 30	6 x 6 = 36				
1 x 7 = 7	2 x 7 = 14	3 x 7 = 21	4 x 7 = 28	5 x 7 = 35	6 x 7 = 42	7 x 7 = 49			
1 x 8 = 8	2 x 8 = 16	3 x 8 = 24	4 x 8 = 32	5 x 8 = 40	6 x 8 = 48	7 x 8 = 56	8 x 8 = 64		
1 x 9 = 9	2 x 9 = 18	3 x 9 = 27	4 x 9 = 36	5 x 9 = 45	6 x 9 = 54	7 x 9 = 63	8 x 9 = 72	9 x 9 = 81	

外层循环控制行(第二个数)

内层循环控制列(第一个数)



练习

基于嵌套循环完成如下需求



- 需求1: 根据输入的直角边的边长，打印等腰直角三角形（如下为直角边为5的等腰直角三角形）

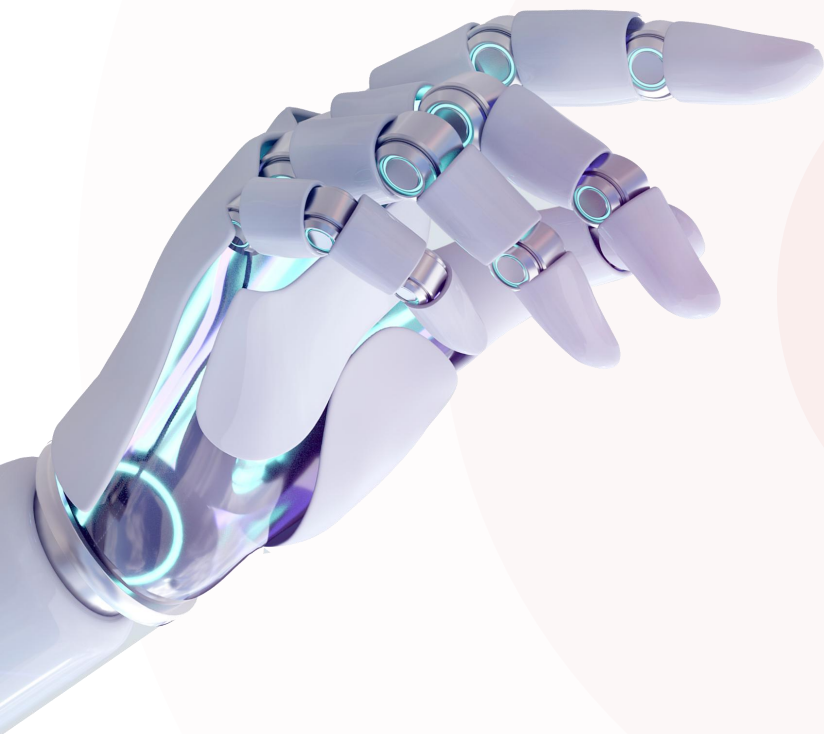
```
*
* *
* * *
* * * *
* * * * *
```

- 需求2: 根据输入的数字，打印对应的数字金字塔

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
```

- 需求3: 打印国际象棋棋盘

```
■ □ ■ □ ■ □ ■ □
□ ■ □ ■ □ ■ □ ■
■ □ ■ □ ■ □ ■ □
□ ■ □ ■ □ ■ □ ■
■ □ ■ □ ■ □ ■ □
□ ■ □ ■ □ ■ □ ■
■ □ ■ □ ■ □ ■ □
□ ■ □ ■ □ ■ □ ■
```

3

循环

- ◆ while循环
- ◆ for循环
- ◆ 嵌套循环
- ◆ 案例



案例

根据需求完成如下案例



- 需求：根据输入的用户名密码执行登录操作，具体要求如下：
 1. 正确的用户名和密码为admin/666888 、zhangsan/123456、taoge/888666
 2. 输入用户名和密码进行登录，直到登录成功，程序结束运行；如果登录失败，则继续输入用户名和密码进行登录
 3. 输入的用户名和密码不能为空！
 4. 登录成功：输出 "登录成功，进入B站首页~"
 5. 登录失败：输出 "用户名或密码错误, 请重新输入!"

BRIEF 小结

- break关键字的作用？
 - 不能够单独书写，只能出现循环中，表示结束、跳出的意思。
- continue关键字的作用？
 - 不能够单独书写，只能出现循环中，表示中断本次循环，直接进入下一次循环。



练习

根据需求完成如下练习



需求：用户名密码登录，正确的用户名和密码为admin/666888 、zhangsan/123456、taoge/888666, 5次登录机会，输入错误五次，不允许再操作了。



案例

猜数字游戏



1. 系统随机生成一个随机数

```
import random
```

```
random_number = random.randint(a: 1, b: 100)
```

2. 用户根据提示猜数字，并将所猜的数字输入系统

3. 如果猜错，系统给出提示是猜大了，还是猜小了，然后继续输入猜的数字

4. 如果猜对，系统自动退出，游戏结束



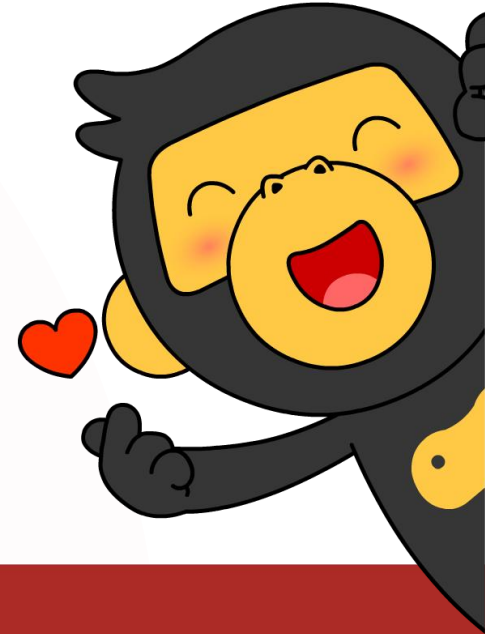
练习

完成如下需求



需求1：将1-1000之间（含1000）所有的5的倍数的数字累加起来。

需求2：统计字符串 "akiwksjakdiklowiqaaamnvbamvaxnsjdsjkaaxkjd" 字符串中有多少个a和k。



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌